

ProcGen Context Studio

Advanced Enterprise Configuration & Rulebook Guide

Version 2.0 | Comprehensive Reference for Procurement Admins

The Omni-Agent Context Studio is the central nervous system for your ProcGen platform. It allows category managers and enterprise admins to inject precise behavioral guardrails, financial parameters, and strict compliance boundaries directly into the AI agents (Niti, Tark, Anveshan) without writing backend code.

1. How Context Logic Impacts the AI

Unlike traditional software, Large Language Models (LLMs) rely on prompt injection to shape their behavior. When you deploy a YAML file in the Context Studio, the system dynamically recompiles the "System Prompt" for the target agent.

Impact: If you change a rule from "Request Net 30" to "Demand Net 60", the agent will instantly adapt its negotiation strategy globally on the very next message it generates. No server restarts or software deployments are required.

2. Defining Scopes and Financial Parameters

The "scope" section of your YAML file acts as the mathematical and geographical boundary for the agent. Below is an exhaustive list of parameters you can define and how they impact the AI:

target_discount_percentage: [Integer] — The baseline mathematical goal. If set to 20, Niti will scrape live market data, calculate a 20% discount, and use that as her opening anchor.

max_budget_buffer: [Integer] — The absolute ceiling. If a vendor refuses to drop below this percentage (e.g., 5% over market price), Niti is hard-coded to walk away or escalate to a human.

approved_regions: [Array] — Restricts the AI. If set to [North America], Anveshan will automatically filter out any suppliers headquartered in EMEA or APAC during sourcing.

required_compliance: [Array] — e.g., [SOC-2, ISO-27001]. Tark will automatically reject or redline any vendor contracts that do not explicitly contain clauses guaranteeing these certifications.

3. Writing Bulletproof Rules

The "rules" array is where you define the natural language constraints. AI agents are highly literal. Vague rules lead to hallucinations; precise rules lead to clinical execution.

'L WEAK RULE (Vague Impact):

- "Try to be aggressive and get a good deal on payment terms."

Result: The AI might accept Net 30 because it thinks that is a "good deal".

' STRONG RULE (Clinical Impact):

- "Never accept the initial offer. Demand Net 60 terms. If the vendor refuses Net 60 three times in a row, escalate to a human manager immediately."

Result: Niti's state-machine is explicitly programmed. She will rigorously count vendor rejections and gracefully exit the chat to a human exactly on the third strike.

4. Enterprise Playbook Examples

Below are advanced, production-ready examples you can copy directly into the Context Studio.

Playbook A: IT SaaS Renewals (Agent: Niti)

```
definition: "SaaS Enterprise Renewal Protocol"
description: "Strict guardrails for negotiating cloud software licenses."
scope:
  target_discount_percentage: 25
  max_budget_buffer: 0
  required_compliance: [SOC-2 Type II, GDPR]
rules:
  - "Always request a 3-year lock-in in exchange for a 25% discount."
  - "If the software handles PII, demand a localized data residency clause."
  - "Never agree to auto-renewing contracts. Demand manual 60-day notice."
```

Playbook B: Green Sourcing (Agent: Anveshan)

```
definition: "ESG-First Supplier Discovery"
description: "Filters the global supply chain for sustainable vendors."
scope:
  approved_regions: [North America, Western Europe]
  max_carbon_footprint_tier: "Tier 2"
rules:
  - "Automatically disqualify any vendor lacking a public ESG report."
  - "Prioritize vendors with localized distribution to reduce freight emissions."
```

5. The Safety Lifecycle

1. Bootstrap: Write your YAML securely. The syntax highlighter will catch basic structural errors.
2. Simulate: The system internally evaluates your parameters against edge-case variables before allowing execution.
3. Deploy: Pressing Deploy flushes the AI cache. Old logic is destroyed; the new rulebook takes over immediately.
4. Observe: Future telemetry will track "Drift"—alerting you if an agent deviates from these exact

YAML boundaries.